

ZERO TO PRODUCTION · PHASE 2

THE ULTIMATE HACKATHON

CinemaSeat

When Everyone Wants the Same Seat

LEARN IT > BUILD IT > SHIP IT

PROBLEM STATEMENT

Saturday, 8 August 2026 | 9:00 AM to 5:00 PM | CUET Campus

IEEE Computer Society, CUET Student Branch Chapter

In partnership with Poridhi.io

The night everything sold out (except it didn't)

Zayan has been waiting months for this. *Spider-Man: Brand New Day* hits theatres tomorrow, and the midnight premiere seats just went live at 8 PM sharp.

He opens **CinemaSeat**, the movie ticketing app everyone in his friend group uses. On a normal Friday he can pick a show, choose seats, pay, and have his QR ticket in under two minutes. Tonight feels different before he even taps anything. The homepage is sluggish, the countdown feels louder than usual, and his group chat is already filling with “is it live yet?” messages.

At 8:00:01 he lands on the premiere showtime. Seat F12, middle of the row, perfect view, still shows available. He taps it. Held. Almost there.

Then the OTP never arrives.

One minute. Two. The hold timer keeps ticking. He retries. The seat map freezes. Sometimes payment starts and dies halfway. Sometimes the app says the seat is his, then a refresh shows it gone. After enough failed attempts he gives up, frustrated and ticketless.

He isn't alone. Within the hour, social media is full of the same story: missing OTPs, failed payments, seats that appear free and then vanish, and a few unlucky people who somehow got confirmation for seats that were already sold. What should have been a fun premiere night turned into a queue of angry fans refreshing a spinner.

Why this matters

Blockbuster releases create traffic spikes that everyday systems are not built for. When a huge title opens advance booking, thousands of people rush the same showtimes at the same second. Popular seats become tiny battlegrounds. If the system is not careful you get slow pages, failed checkouts, and worse, the same seat confirmed for more than one person.

Your job is to build a **scalable, reliable movie ticket booking system** that stays usable under pressure and **never sells the same seat twice**.

What you are building

Design and implement a cinema ticketing platform that can:

- Let users browse movies, showtimes and theatres
- Show a live seat map for a show
- Hold a seat, complete payment, and confirm a booking
- Release a hold automatically if payment is not completed in time
- Survive heavy concurrent demand without double-booking
- Integrate with the provided payment and OTP gateway (see below)
- Be containerized, tested, and deployable with a sensible DevOps setup

You do not need a cinema admin portal. Pre-populate the database with movies, theatres, showtimes, seat layouts and prices.

Language, framework, database and cloud platform are your choice unless a rule says otherwise.

You can work on milestones in parallel. You do not need to finish Implementation before starting the DevOps Pipeline. In fact, you should not.

The provided gateway

Zayan's OTP never arrived, and his payment died halfway. That was not a made-up detail. We are giving you a payment and OTP service that behaves exactly like a real one, which is to say badly.

Do not mock this yourself. Every team integrates with the same container, so every team faces the same conditions.

Getting it

```
# add to your docker-compose.yml
gateway:
  image: asifmahmoud414/mock-gateway:latest
  ports: ["9000:9000"]
```

Source and Dockerfile are also in the starter repo if you would rather build it locally. **Pull the image before the event.**

Pull the Gateway service image from dockerHub (asifmahmoud414/mock-gateway:latest)

Endpoints

```
POST /charge      { amount, currency, booking_ref, callback_url }
                  -> 202 { payment_id, status: "PENDING" }

POST /refund      { payment_id }
                  -> 202 { status: "PENDING" }

POST /otp/send    { phone, ref } -> 202
POST /otp/verify  { ref, code }  -> 200 | 400

GET /health       -> 200
```

The gateway calls **you** back at the callback_url you supply:

```
{ "event_id": "evt_001", "payment_id": "pay_xyz",
  "booking_ref": "bk_001", "status": "SUCCEEDED", "amount": 450 }
```

status is one of SUCCEEDED, FAILED, REFUNDED.

Documented misbehaviour

This is the specification, not a bug list. Design for it.

BEHAVIOUR	RATE
Callback delayed 2 to 15 seconds	Always
Payment fails (FAILED)	10%
The same callback delivered twice	8%
/charge returns 500 or times out	2%
OTP delayed or never delivered	10%

Three consequences worth thinking about before you write code:

1. Your /pay handler cannot wait for the gateway. It must return quickly and let the callback finish the job.
2. Always return 200 from your callback handler, even for a duplicate. A non-200 tells the gateway that delivery failed, and it will retry forever.
3. A duplicate callback must not create a second payment, must not confirm the booking twice, and must not double-count revenue.

Control headers

Send these on /charge while building and testing.

HEADER	EFFECT
X-Mock-Mode: deterministic	2 second delay, always succeeds, no duplicates
X-Mock-Force: fail	Guaranteed failure
X-Mock-Force: duplicate	Guaranteed duplicate callback
X-Mock-Force: timeout	Guaranteed timeout on /charge
X-Mock-Force: race	Callback arrives before /charge returns
X-Mock-Force: success	Guaranteed clean success

IMPORTANT

Judges will use the force headers, so every team is tested on identical conditions rather than on luck. Deterministic mode is for building. Turn it off before you believe anything.

Judging hooks

Almost every design choice is yours. These four things are not, because judges need to verify your system quickly and identically across all teams.

1. GET /health returns 200 in under one second, and keeps doing so when the gateway is down.

2. `HOLD_TTL_SECONDS` is read from the environment, not hardcoded. Judges will run your stack with a short value to watch a hold expire.
3. Your README lists the exact request for holding a seat and for fetching a seat map. Judges will point tests at these.
4. docker compose up works from a clean clone with no manual steps.

Everything else, including your endpoint names, schema, service boundaries and internal design, is yours.

Milestones

Milestone 1: System Design

Map out the services (or modules) your ticket booking system needs. Architecture is the backbone here. Under a premiere rush, many users may fight for the same seat in a very short window.

Your design should show how the system stays robust under high request volume, and how one failing part does not take everything down with it. Prepare an architecture diagram, and keep data models and API design simple and clear.

Keep it simple. A fancy diagram you cannot explain is worse than a simple design you can defend. Splitting into services is a choice, not a requirement. If you split, be ready to say what it cost you. If you did not split, be ready to say why you did not need to.

Feeds: System Architecture (25)

Milestone 2: Implementation

Turn the design into a working system.

- Integrate the provided gateway. Do not write your own mock.
- A polished UI is optional. A minimal frontend showing browse, seat select, hold, pay, confirm is enough. Visual polish does not earn extra marks.
- Implement only the endpoints you actually need.
- Handle inter-service communication if you split services.
- Expose one base URL to the frontend.
- Write unit tests for your core logic, especially the concurrency and duplicate-callback paths. Integration tests are a plus.
- Dockerize everything. docker compose up must bring the whole stack up with no external dependencies.

Focus on the core booking path and correctness under concurrency, not on every cinema feature you can imagine.

Feeds: Functionality (25), Code Quality and Testing (15), Containerization (part of 15)

Milestone 3: DevOps Pipeline

Make the project shippable.

- Provision on Poridhi VM or AWS (see Deployment below).
- Set up CI/CD with GitHub Actions or equivalent:

- CI runs on pull requests and pushes to the default branch
- Code does not merge without passing CI
- CD runs only on pushes to the default branch
- Change-aware workflows are a plus: test and deploy only what changed
- Keep services reachable during deployment where possible.

Include a pipeline diagram in your documentation.

Feeds: Containerization and CI (15), Deployment (10)

Milestone 4: Prove It

Claims are worth nothing. Numbers are worth marks.

Scenario A is required. Scenario B is required. Scenario C is bonus.

Scenario A: One seat, many buyers

Pick one seat on one showtime. Fire **100 concurrent hold requests** for that exact seat in a single burst.

Report: requests sent, successful holds, rejections, and oversell count.

Oversell must be zero. Exactly one request may succeed. Ninety-nine must be cleanly rejected. Then fetch the seat map and confirm the seat is held once, not twice.

A test that spreads users across many seats does not count. The seats must fight. Two hundred users across a hundred seats will show you zero collisions and tell you nothing.

Scenario B: The abandoned hold

Hold a seat and walk away without paying. Wait for the hold to expire.

Report: the timeline you observed, and evidence that the seat returned to available and was then successfully booked by a different user.

Run this with a short `HOLD_TTL_SECONDS` so it completes in under a minute.

Scenario C (bonus): Find your breakpoint

Ramp virtual users on your seat map and hold endpoints until the system degrades.

Report: where p95 latency turns upward, where errors begin, and **your explanation of what the bottleneck was**. Connection pool? Database contention? Blocked event loop? Memory?

The explanation is what earns the marks. The number on its own tells us nothing.

Two things about load testing

We are not comparing throughput numbers between teams. You are deploying to different instance types with different resources. Team A reporting 400 requests per second and Team B reporting 120 tells us about their hardware, not their engineering. We judge your methodology, your breakpoint, and your explanation of the bottleneck. Never the raw magnitude. **Your VM size is not your engineering.**

Do not run the load generator on the same machine as your application. If k6 and your app compete for the same 2 vCPUs, you are measuring your load tool fighting your own service. Run it from your laptop against the deployed URL, or from your host against your local containers.

Feeds: Functionality (25), Code Quality and Testing (15)

Deployment

You have two options. **Both are fully acceptable and neither scores higher for the Deployment criterion.** Judges check that it is deployed and reachable, not where.

Option 1: Poridhi VM plus load balancer. The simpler path. Fewer moving parts, less that can go wrong, faster to get live.

Option 2: AWS. The harder path. More to learn, more to break, and continuous deployment genuinely matters here. Attempting AWS successfully earns bonus marks.

Your lab is a clock, so start it right

Your Poridhi lab, including AWS credentials, runs for 12 hours from launch. There is no idle timeout, but there is no extension either. When the lab ends, the VM and the AWS account both disappear.

- Launch your lab at 9:00 AM, during the opening session. Not at 8:30. Not at 11:00. Twelve hours from 9:00 covers the event and the judging window that follows.
- Designate one infrastructure owner. That person's lab hosts the team's deployment and is not stopped, restarted, or used for experiments. Other members can use their own labs freely.
- Your infrastructure is disposable. If it vanishes, only what is in your repository survives. Your deployment must be reproducible from a clean clone. No hand-configured servers, no manual steps you did not write down.

Bonus tasks

Worth up to +10 marks. Attempt these only when the required milestones are solid. A half-built bonus is worth less than a finished requirement.

- **Fault isolation.** With the gateway container stopped completely, browsing, seat maps and holds still work, /health stays green, nothing returns 500, and pending payments recover when the gateway comes back.
- **Monitoring and observability.** Structured logs with request IDs, a metrics endpoint, distributed tracing.
- **Graceful degradation under peak.** People browsing other movies still get a usable experience while the premiere showtime is being hammered.
- Nginx reverse proxy and load balancing.
- **Security basics.** Authentication, authorization, rate limiting, input validation. Verifying the gateway callback signature counts here.
- AWS deployment, as described above.
- Scenario C from Milestone 4.

Deliverables

There are no slides. Your documentation is your presentation.

Submit a single public GitHub repository containing the following along with your source codes:

FILE	CONTENTS
README.md	What you built, what works, what does not. Architecture diagram. How to run locally from clone to docker compose up. Your deployed URL. The exact requests for holding a seat and fetching a seat map.
DECISIONS.md	Three decisions your team genuinely argued about. For each: the options you considered, what you chose, why, and what you gave up.

Both artifacts are judged: `docker compose up` from a clean clone, and your deployed URL.

General instructions

- Start a new GitHub repository after the opening session. It must be public at submission.
- Build during the hackathon. Work prepared in advance, even partially, is not accepted. Environment setup, tooling, and account configuration beforehand are expected and encouraged.
- Standard scaffolding is fine: framework generators, starter templates, component libraries, open-source packages. Attribute anything you pull in.
- Do not push to the default branch after code freeze.
- Commit early and often. Your history is the record of how your system took shape.
- Unfair play, misconduct or rule breaches may lead to disqualification. Organisers have the final say, and you will be notified if rules change in any meaningful way.

KEEP PERSPECTIVE

This is a lot for eight hours. Nobody expects a perfect cinema platform.

Focus on the core path: seats, booking correctness, containers, and a deployable demo. A smaller system that never double-books and is properly shipped beats a larger one with a race condition.

One line to remember

Build a movie booking system that stays calm when Brand New Day drops,

and never sells the same seat twice.